

Yep 😊 — this is the missing piece behind **why we need salt**. Let's make it visual.

🌈 What is a rainbow-table attack?

Imagine an attacker steals a database containing password hashes:

email	passwordHash
gopi@gmail.com	2cf24d...
user2@gmail.com	5e8848...
user3@gmail.com	482c81...



They **don't know the passwords**.

But here's the trick:

Instead of calculating hashes from scratch after stealing the database, an attacker can **prepare a huge table beforehand**.

1. Without salt

Suppose a website stupidly uses:

SHA256(password)



Someone could build a table like:

Password	SHA-256 Hash
password	5f4dcc3b...
123456	e10adc39...
hello123	...
qwerty	...
letmein	...
admin123	...



This is a **precomputed table**.

The attacker has already done the expensive part:

password
↓
SHA-256
↓
hash



for millions of common passwords.

Then they steal your database and see:

```
gopi@gmail.com → ef92b778...
```



They simply search:

```
ef92b778...
```



in their table.

If they find:

```
ef92b778... → hello123
```



🎯 They know the password.

2. Why is this called "precomputed"?

Because the attacker did this:

BEFORE stealing your database

```
password  
123456  
qwerty  
hello123  
admin  
...  
↓  
SHA-256 each one  
↓  
Huge table
```



Then:

AFTER stealing database

```
Stolen hash  
↓  
Look it up  
↓  
Password found
```



They don't need to calculate every possibility again.

That's **precomputation**.

3. So what's a rainbow table?

A **rainbow table** is a more sophisticated type of precomputed lookup structure designed to reduce the storage needed for huge numbers of password/hash possibilities.

You don't need to understand the mathematical details of how the chains work yet.

For your interview-level understanding:

A rainbow table is a precomputed structure attackers can use to recover passwords from hashes, particularly when hashes are unsalted.

That's enough.

4. Now SALT destroys this advantage

Suppose Gopi has:

```
password = hello123
```



The server generates:

```
salt = X7A91
```



Then:

```
hello123 + X7A91
  ↓
Argon2id
  ↓
HASH_A
```



Another user:

```
password = hello123
salt = P82KD
```



gets:

```
hello123 + P82KD
  ↓
Argon2id
  ↓
HASH_B
```



So:

```
Gopi   → HASH_A
User 2 → HASH_B
```



Even though:

```
Gopi's password = hello123
User 2's password = hello123
```



5. Why can't the attacker just make a rainbow table anyway?

They could try.

But now they'd need to account for the salt.

Imagine there are:

```
1,000,000 common passwords
```



and:

```
millions of possible salts
```



The combinations become enormous.

And importantly, the attacker generally **doesn't know the salt until they obtain the stored password record**, so a universal precomputed table made in advance doesn't work the same way.

The attacker now has to perform password guesses **for the particular salt**.

And because we're also using Argon2id:

```
password + salt
  ↓
```



Argon2id



hash

each guess is deliberately expensive.

So we get **two layers of protection**:

Salt



prevents useful universal precomputed tables

Argon2id



makes each individual password guess expensive



That's the connection.

6. Simple analogy

Imagine the attacker wants to open millions of lockers.

Without salt

Every locker uses the same lock mechanism.

The attacker can prepare:

Key A → Lock result A

Key B → Lock result B

Key C → Lock result C

...



Then when they encounter a stolen lock, they already have the answers.

With salt

Every locker has a **different random modification**:

Locker 1 → random modification A

Locker 2 → random modification B

Locker 3 → random modification C



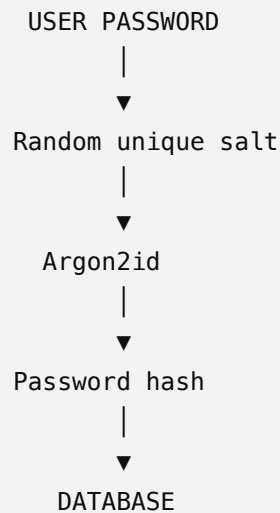
The attacker's precomputed universal table becomes much less useful.

And Argon2id makes testing each key against each locker expensive.



The whole password-security story now connects

This is what we've been building toward:



Why?

Plaintext password ❌

Database leak → immediate password exposure.

SHA-256 without salt ❌

Fast guessing + precomputed/rainbow tables.

SHA-256 + salt

Much better against precomputed tables, **but still too fast for modern password storage.**

Argon2id + unique salt ✅

```
graph TD; A[Salt] --> B[prevents useful precomputed tables]; C[Argon2id] --> D[makes guessing expensive]; E[Hash] --> F[we don't store the original password];
```

That's the mental model. 🗝️

And honestly, **now you understand the "why" behind password hashing**, rather than just memorizing "use Argon2id" — which is exactly what you want for interviews.



yes

Awesome 🥳🔥 Then let's move back to the **actual Next.js authentication flow**.

We have now completed the password side:

```
Password
  ↓
Argon2id + unique salt
  ↓
Password hash
  ↓
Database
```



Now we move to the **session side**.

7.4.11 — What exactly is a session?

This is the next piece that usually confuses people.

After successful login, we have:

```
User
  ↓
email + password
  ↓
password verified ✅
```



But here's the problem:

What happens on the NEXT request?

Suppose the user now visits:

```
/dashboard
```



The browser doesn't want to send:

```
email + password
```



again.

And we definitely don't want the server to repeatedly authenticate the password.

Instead, we create a **session**.

Think of a session as a temporary "logged-in relationship"

After login:

```
User 42
  |
  | creates
  ▼
Session ABC123
```



The server/database stores:

sessionId	userId	expiresAt

ABC123	42	tomorrow



Then the browser gets:

```
Cookie:
sessionId=ABC123
```



So now:

```
Browser
  |
  | Cookie: sessionId=ABC123
  ▼
Server
  |
  | "Who owns ABC123?"
  ▼
Database
  |
  | userId = 42
  ▼
Server
```



Now the server knows:

```
"This request belongs to User 42."
```

Why don't we put `userId=42` directly in the cookie?

You *could* technically send something like:

```
userId=42
```



But that doesn't prove anything.

A user could modify their cookie:

```
userId=42
```



to:

```
userId=43
```



and say:

```
"Hey, I'm user 43 now." 😂
```

So the server needs a value that the user **cannot simply change to impersonate another user**.

That's why we use an unpredictable session identifier:

```
sessionId=8f5d9e3a7c...
```



The server controls the mapping:

```
8f5d9e3a7c... → User 42
```



What if the attacker steals the session cookie?

This is important.

If someone obtains:

```
sessionId=ABC123
```



they may be able to act as that user's session until it expires or is revoked.

This is called **session hijacking**.

That's why session IDs must be:

- unpredictable
- generated securely
- transmitted over HTTPS
- protected with `HttpOnly`
- appropriately scoped with cookie settings
- expired/revoked

And that's also why protecting against XSS matters: malicious JavaScript shouldn't be able to simply read an `HttpOnly` session cookie.

Session vs password

This distinction is worth memorizing:

Password	Session
Used to authenticate login	Used to maintain logged-in state
User knows it	User doesn't need to know session ID
Stored as a hash	Session record stored server-side in this model
Sent during login	Cookie sent on later requests
Long-lived credential	Usually temporary
Password verification is expensive	Session lookup should be relatively cheap

So:

```
LOGIN
password
  ↓
verify
  ↓
create session

AFTER LOGIN
session cookie
  ↓
identify session
```



↓
identify user

One subtle thing: "session" doesn't necessarily mean database

You'll often hear:

"Session-based authentication means storing sessions in a database."

That's a common implementation, but not the definition.

A session is fundamentally the **server's representation of an authenticated user's state**.

You could store session state in:

Database
Redis
another server-side store



The important thing is that the server can use the session identifier to determine the authenticated user.

And this leads directly into 7.5

Now we're ready for the interesting question:

What happens when a Server Component needs to know who the current user is?

For example:

```
</> TypeScript

export default async function Dashboard() {
  const user = await getCurrentUser();

  return <h1>Hello {user.name}</h1>;
}
```



Where does `getCurrentUser()` get the session from?

Server Component



```
cookies()  
  ↓  
sessionId  
  ↓  
session store / DB  
  ↓  
user
```

That's **Lesson 7.5 — Server Components + Authentication**, and that's where the authentication concepts we've built start connecting directly to the Next.js architecture you've already learned. 🔥

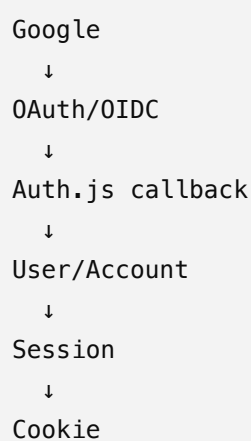




Awesome 🤗 Let's move to the **next important Auth.js concept**: how the session is **actually used to protect pages and APIs**.

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:



```
graph TD; A[Google] --> B[OAuth/OIDC]; B --> C[Auth.js callback]; C --> D[User/Account]; D --> E[Session]; E --> F[Cookie];
```

Google
↓
OAuth/OIDC
↓
Auth.js callback
↓
User/Account
↓
Session
↓
Cookie



Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/  
├─ login/  
│   └─ page.tsx  
└─ dashboard/  
    └─ page.tsx
```

Our dashboard should only be accessible to logged-in users.

We can write:

```
<\/> TypeScript  
  
// app/dashboard/page.tsx  
  
import { auth } from "@auth";  
import { redirect } from "next/navigation";  
  
export default async function DashboardPage() {  
  const session = await auth();  
  
  if (!session) {  
    redirect("/login");  
  }  
  
  return (  
    <div>  
      <h1>Welcome {session.user?.name}<\/h1>  
      <p>{session.user?.email}<\/p>  
    <\/div>  
  );  
}
```

Now let's understand **exactly what happens**.

2. User requests `/dashboard`

Browser:

```
</> http

GET /dashboard
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript

const session = await auth();
```



runs.

3. What does `auth()` do?

Remember where it came from:

```
</> TypeScript

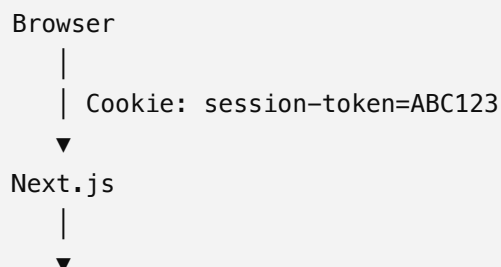
// auth.ts

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({...});
```



So `auth()` is an Auth.js server-side function.

Conceptually:




```
Dashboard Server Component
  |
  | auth()
  ▼
Auth.js
  |
  ▼
Session verification / lookup
  |
  ▼
User session
```

If the session is valid:

</> TypeScript



```
session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

</> TypeScript



```
const session = await auth();
```

returns something equivalent to:

</> TypeScript



```
null
```

So:

</> TypeScript



```
if (!session) {
  redirect("/login");
}
```

```
}
```

executes.

Next.js sends a redirect response.

Conceptually:

```
</> http
```



```
307 Temporary Redirect
```

```
Location: /login
```

The browser then requests:

```
</> http
```



```
GET /login
```

So:

```
User → /dashboard
```

```
↓
```

```
auth()
```

```
↓
```

```
no session
```

```
↓
```

```
redirect("/login")
```

```
↓
```

```
/login
```



5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript
```



```
if (!session)
```

That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

</> TypeScript



```
const session = await auth();

if (!session) {
  redirect("/login");
}

if (session.user.role !== "ADMIN") {
  redirect("/forbidden");
}
```

Now we're asking two different questions:

Authentication:

"Who are you / are you logged in?"



Authorization:

"Are you allowed to do this?"

6. But there's an important security rule

Never rely only on hiding UI.

For example:

</> TypeScript



```
{session?.user.role === "ADMIN" && (
  <button>Delete User</button>
)}
```

This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http  
  
POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```



We can do:

```
</> TypeScript  
  
import { auth } from "@auth";  
import { NextResponse } from "next/server";  
  
export async function POST(request: Request) {  
  const session = await auth();  
  
  if (!session) {  
    return NextResponse.json(  
      { error: "Unauthorized" },  
      { status: 401 }  
    );  
  }  
  
  // User is authenticated  
  // Continue processing order...  
  
  return NextResponse.json({  
    success: true,  
  });  
}
```



Now:

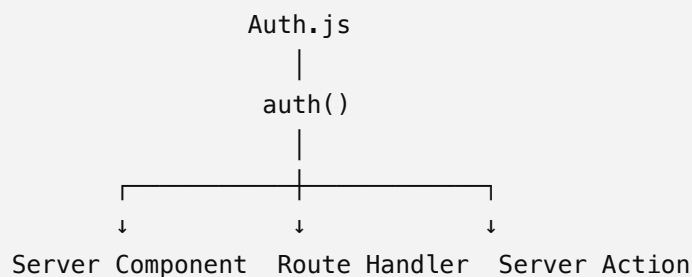
```
Browser  
|
```



```
POST /api/orders
Cookie: session-token=ABC
▼
Next.js Route Handler
|
▼
auth()
|
├── no session → 401
|
└── valid session → continue
```

8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

Server Component

`</>` TypeScript

```
const session = await auth();
```

Route Handler

`</>` TypeScript

```
const session = await auth();
```

Server Action

`</>` TypeScript

```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

```
</> TypeScript   
"use server";  
  
import { auth } from "@auth";  
  
export async function deleteAccount() {  
  const session = await auth();  
  
  if (!session) {  
    throw new Error("Unauthorized");  
  }  
  
  // Delete account...  
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer 🎯

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:

1. LOGIN
 `signIn()`
 ↓
 `OAuth`
 ↓
 `callback`
 ↓
 `session created`
2. READING SESSION
 `auth()`
 ↓
 `"Who is currently logged in?"`
3. PROTECTING ACTIONS
 `auth()`
 ↓
 `authenticate`
 ↓
 `authorize`
 ↓
 `perform operation`

That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

⌄ TypeScript

```
callbacks: {  
  async jwt() {},  
  async session() {},  
  async signIn() {},  
}
```

```
async redirect() {}  
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥



how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

🔑 Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	🛡️ Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions
7.7	🔄 Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	👤 Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session
7.9	🔑 Authorization / RBAC	Roles, permissions, admin/user access
7.10	🍪 Session Strategies	Database sessions vs JWT sessions
7.11	🔒 CSRF + SameSite	Why state-changing requests need protection
7.12	🌐 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	📄 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions

#	Topic	What you'll learn
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

- 1. **Callbacks**
- 2. **Session customization**
- 3. **Authorization/RBAC**
- 4. **JWT vs DB sessions**
- 5. **CSRF/SameSite**
- 6. **Middleware/Proxy**

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 👍 **After we finish those remaining Auth.js topics, we can consider the Authentication & Authorization section complete.**

More precisely, your **Next.js Auth & Authorization roadmap** will cover:

AUTHENTICATION



- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT

- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:

Browser

↓

Login

↓

Auth.js

↓

Google OAuth/OIDC

↓



```
Callback
  ↓
Google token exchange
  ↓
User + Account
  ↓
Adapter
  ↓
Database
  ↓
Session
  ↓
Cookie
  ↓
Protected request
  ↓
auth()
  ↓
Authentication
  ↓
Authorization
  ↓
Protected resource
```

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅



ChatGPT can make mistakes. Check important info.